

Specifikace softwarového díla a časový plán implementace pro: **Simple code generation toolkit**

Jakub Turek

4.4.2026

Abstrakt

Projekt zaměřující se na implementaci jednoduché sady nástrojů pro generování kódu na základě projektem navrženého mezikódu (IR) v programovacím jazyce Haskell. Primárním cílem je prozkoumat různé techniky code generation a vytvořit nástroj, který je uživatelsky jednodušší než LLVM, ale zároveň poskytuje dostatečné možnosti, aby mohl být využit jako backend pro plnohodnotný překladač, nebo pro výuku teorie překladačů.

Obsah

1	Základní informace	4
1.1	Popis a zaměření softwarového díla	4
1.2	Použité technologie	4
1.3	Konvence tohoto dokumentu	4
2	Stručný popis softwarového díla	4
2.1	Důvod vzniku softwarového díla a jeho základní části a cíle řešení	4
2.2	Hlavní funkce	4
2.3	Motivační příklad užití	5
2.4	Prostředí aplikace	5
2.5	Omezení díla	5
2.6	Dokumentace díla	5
3	Vnější rozhraní	5
3.1	Uživatelské rozhraní, vstupy a výstupy	5
3.2	Rozhraní s hardware	6
3.3	Rozhraní se software	6
4	Detailní popis funkcionality	6
4.1	Mezikód a jeho parsování	6
4.2	Code generation a optimalizace	7
5	Ostatní (mimofunkční) požadavky	7
5.1	Požadavky na rozšiřitelnost a začlenitelnost	7
6	Ostatní požadavky	7
7	Negativní vymezení	8
8	Time-line & Milestones	8
9	Zakončení	9
9.1	IR	9
9.2	Implementace code generation	10
9.3	Demo	11
A	Vymezení pojmů	12

Tabulka revizí

Jméno	Datum	Důvod změny	Verze
Jakub Turek	27.8.2026	Úprava specifikace po konzultaci: přidání RISC-V jako primární cílové platformy, integrace s frontendem přesunuta do BP	1.1
Jakub Turek	27.8.2026	Přidání shrnutí současného stavu implementace	1.2

1 Základní informace

1.1 Popis a zaměření softwarového díla

Software vzniklý v rámci projektu bude jednoduchý backend překladače, doplněný o definici vlastního mezikódu (IR), který bude sloužit jako jeho vstup a interface pro komunikaci s frontendem. Backend bude schopen pro vstup v podobě tohoto mezikódu vygenerovat ekvivalentní RISC-V/x86 assembly (primární cílovou architekturou aktuálního prototypu je RISC-V, podpora pro x86 bude přidána později pro demonstraci rozšiřitelnosti projektu). V tomto ohledu by měl být kladen důraz hlavně na snadnou rozšiřitelnost, tedy aby přidání nové cílové architektury vyžadovalo, co nejmenší zásah do již existujícího kódu. Z vygenerovaného assembly bude následně pomocí externího assembleru a linkeru pro konkrétní platformu vygenerován spustitelný soubor. Za alternativu projektu je možné považovat LLVM, které nabízí vlastní IR a rozsáhlou na něj navázanou infrastrukturu pro tvorbu překladačů, primárně jejich backendů. Předchozí práce s LLVM při tvorbě vlastního překladače, pak byla i hlavní motivací pro vznik celého projektu. Kdy jsem chtěl tuto externí závislost na robustním, ale komplexním nástroji jako je LLVM nahradit vlastní alternativou. Z čehož později po konzultaci s vedoucím projektu vzešla myšlenka vytvořit zjednodušenou alternativu právě k LLVM, která by sloužila pro podobné projekty, pro než je LLVM příliš komplexním nástrojem, případně nedokážou plně využít všech jeho benefitů.

1.2 Použité technologie

Pro implementaci backendu bude využit programovací jazyk Haskell. Dále bude projekt využívat externí assembler a linker, pro převod vygenerovaného assembly do podoby objektových/spustitelných souborů pro konkrétní platformu/operační systém.

1.3 Konvence tohoto dokumentu

Názvy externích závislostí a knihoven jsou v dokumentu zvýrazněny kurzívou. V kapitole shrnující současný stav implementace jsou pak kurzívou zvýrazněny také názvy typových tříd deklarovaných v rámci implementace. Tučně jsou pak zvýrazněny názvy jednotlivých IR instrukcí. V podkapitole pojednávající o demo ukázce projektu jsou tučně zvýrazněny jednotlivé přepínače konzolového rozhraní.

2 Stručný popis softwarového díla

2.1 Důvod vzniku softwarového díla a jeho základní části a cíle řešení

Jak již bylo řečeno úvodem, primárním impulzem ke vzniku projektu byla práce s LLVM v rámci jiného projektu. Následovaná snahou primárně ze studijních důvodů LLVM nahradit vlastní implementací. Tato snaha eventuálně vyústila v myšlenku implementovat jednoduchý nástroj, který by nabízel základní funkcionalitu LLVM a mohl by být použit jako jeho alternativa pro obdobné malé a výukové projekty, které většinou využívají jen zlomek z nabízených možností, ale zároveň jsou zatíženy vysokou komplexností celého systému. Podobně jako LLVM bude projekt založen na vlastní IR a nástroji, který na jejím základě vygeneruje odpovídající assembly pro cílovou architekturu (v tuto chvíli RISC-V/x86). Projekt pak bude možné využít jednak pro tvorbu zcela nových překladačů doplněním o frontend generující toto IR pro jazyk, pro nějž chce uživatel vytvořit překladač. Případně bude možné ho integrovat do již existujících projektů úpravou frontendu daného překladače a jím generovaného mezikódu, ať už jde o LLVM IR v případě projektů využívajících LLVM, nebo třeba kombinátory v jazyce C v případě MicroHs.

2.2 Hlavní funkce

Software vyvinutý v rámci projektu by měl sloužit jako nástroj pro generování strojového kódu. Samotný kód bude generován na základě vstupu v podobě souboru s textovou reprezentací IR. V tuto chvíli bude

nástroj schopen generovat instrukce pro platformu RISC-V, případně x86, ale jeho design bude umožňovat pozdější odvození verzí pro další platformy. A při doplnění o vhodný frontend tak bude tvořit překladač daného jazyka pro architekturu RISC-V/x86. Hlavním cílem je tedy vytvořit code generation nástroj, který bude možné využít jako překladačový backend dalšími projekty.

2.3 Motivační příklad užití

Typickým užitím výsledného softwaru je tvorba jednoduchého překladače nějakého programovacího jazyka pro backendem podporované architektury. Pro takovéto využití je třeba projekt doplnit o překladačový frontend daného jazyka generující projektem definovanou IR.

2.4 Prostředí aplikace

Podporovaným prostředím pro využití nástroje budou zařízení s architekturou RISC-V/x86 a operačním systémem Linux, případně Windows (v případě x86 architektury, protože v případě RISC-V aktuálně neexistuje oficiální podpora ze strany Windows). Jako externí závislosti bude pro své fungování program vyžadovat nástroje *as* a *ld*.

2.5 Omezení díla

Výsledný software by neměl být předmětem zásadních omezení v podobě systémových, či licenčních podmínek. Hlavním omezením je v tomto ohledu rozhodnutí delegovat generování objektových souborů a jejich linkování na externí nástroje v podobě *as* a *ld*. Což má za následek nejen závislost projektu na těchto externích nástrojích, ale také nutnost využívat s nimi kompatibilní zápis RISC-V/x86 assembly, nebo nutnost *as* nahradit zcela jiným assemblerem v případě rozšiřování projektu o jím nepodporované architektury.

Méně významným omezením je pak omezení na konkrétní překladačové frontendy, které by měly být s projektem integrovány. Aby tato integrace byla možná, musí konkrétní frontend splnit požadované komunikační rozhraní. Tedy jeho výstupem musí být textová reprezentace IR definované v rámci projektu.

2.6 Dokumentace díla

Jelikož je projekt koncipován jako jednoduchý code generation nástroj, tak bude třeba kromě běžné uživatelské a vývojové dokumentace dodat také specifikaci používané IR. Tato specifikace by zřejmě mohla být součástí běžné uživatelské dokumentace. Zároveň se ale jedná o rozsáhlejší standard definovaný v rámci projektu, jehož znalost je sice nutná pro plnohodnotné využití při vývoji vlastního překladače, nikoliv však pro samotné užití nástroje k vygenerování kódu z již existujících souborů s IR. Pročež se jeví lepším specifikaci tohoto standardu vyjmout do separátního dokumentu. Tento dokument by pak měl společně s uživatelskou dokumentací poskytovat dostatek informací nutných pro využití tohoto projektu k tvorbě vlastního překladače.

3 Vnější rozhraní

3.1 Uživatelské rozhraní, vstupy a výstupy

Interakce s uživatelem bude probíhat prostřednictvím běžného CLI. Program bude nabízet přepínače pro funkcionality běžné u ostatních překladačů jako jsou

- Specifikace výstupního souboru
- Nastavení úrovně optimalizací
- Typ výstupu spustitelný soubor/objektový soubor/assembly

- Specifikace knihoven k linkování

Vstupem programu pak bude soubor obsahující textovou reprezentaci v rámci projektu definované IR. Na základě tohoto vstupu a příslušných přepínačů bude programem vygenerován výstupní soubor v podobě spustitelného souboru programu definovaného vstupním IR, textového souboru obsahující vygenerované assembly, objektového souboru k nalinkování.

3.2 Rozhraní s hardware

Projekt nevyužívá žádný specifický hardware. Zaměřuje se však na generování kódu pro architekturu RISC-V/x86. Projekt tedy v tuto chvíli podporuje pouze zařízení s touto architekturou procesoru, respektive jeho výstupy v podobě spustitelných souborů, či assembly (využívajícího GNU syntaxi v případě x86 assembly) jsou určeny pro tuto platformu.

3.3 Rozhraní se software

Projekt je koncipován jako jednoduchý backend překladače, z čehož plyne nutnost jeho integrace s dalšími kroky překladače kódu. Prvním z rozhraní je interakce s překladačovým frontendem a získání vstupní IR reprezentující překládaný zdrojový kód. Výměna dat na tomto rozhraní bude probíhat prostřednictvím textového souboru, který bude obsahovat jednotlivé IR instrukce zapsané v textové človkem čitelné podobě. Frontend tedy zpracuje překládaný kód a na jeho základě vygeneruje odpovídající IR (instrukce a jejich serializace bude specifikována později), kterou následně zapíše do výstupního souboru. Poté spustí backend, kterému tento soubor předá jako vstup.

Druhým softwarovým rozhraní je rozhraní mezi backendem a assemblerem, který z vygenerovaného assembleru vytvoří objektový soubor ve formátu pro cílový operační systém. Výměna dat na tomto rozhraní bude opět probíhat prostřednictvím dočasného textového souboru, kdy backend opět uloží x86 assembly vygenerované pro vstupní IR do výstupního souboru a následně zavolá externí assembler, kterému předá tento soubor jako vstup.

Posledním softwarovým rozhraním projektu je rozhraní mezi backendem a linkerem, kde opět dojde k výměně dat prostřednictvím souborů. Backend jednoduše spustí externí linker s cestami k souborům, jenž je třeba nalinkovat (objektový soubor vygenerovaný assemblerem a případné knihovny specifikované uživatelem na vstupu), a linker následně zapíše výsledný nalinkovaný spustitelný soubor do backendem specifikovaného výstupního souboru. Toto rozhraní by projekt z koncepčního hlediska pravděpodobně nemusel obsahovat a výstupem backendu by mohl být pouze nenalinkovaný objektový soubor. Avšak z uživatelského hlediska je pravděpodobně přívětivější doplnit backend o schopnost zavolat externí linker a vygenerovaný objektový soubor také nalinkovat.

4 Detailní popis funkcionality

4.1 Mezikód a jeho parsování

V rámci projektu bude definován vlastní mezikód (IR), který bude sloužit jako vstup samotného nástroje. Cílem projektu je vyzkoušet různé alternativní způsoby reprezentace k reprezentaci využívané v rámci projektu LLVM. Aktuálně projekt jako vstupní IR využívá jednoduchou stack-based IR inspirovanou programovacím jazykem Forth a WebAssembly.

Jednou z uvažovaných možností reprezentace je reprezentovat celý překládaný kód pomocí bloků instrukcí a control flow v kódu pak modelovat přímo jako operace nad instruction pointer registrem (tedy jako inkrementaci/dekrementaci instruction counteru o velikost příslušných bloků instrukcí). Takovýto přístup by měl negativní dopad na provádění řady optimalizací používaných právě LLVM a dalšími běžnými překladači, ale mohl by potenciálně zjednodušit IR a reprezentaci control flow v jejím rámci.

V případě, že by se žádný z alternativních přístupů neukázal jako dostatečně efektivní, navrátí by se projekt ke standardně využívané formě reprezentace pomocí mezikódu složeného z basic bloků v static single-assignment formě a reprezentaci control flow pomocí final instruction specifies the jump.

Aby bylo možné s kódem zapsaným pomocí takto definované reprezentace dále pracovat, je nutné v rámci projektu napsat také parser a serializer, který umožní efektivní převádění z a do textového zápisu. Parser by zároveň měl dokázat uživatele smysluplně upozornit na syntaktické chyby v textovém zápisu reprezentace, tak aby bylo co nejsnazší opravit případné chyby v jejím generování.

4.2 Code generation a optimalizace

Klíčovou funkcí je pak samotné generování RISC-V/x86 instrukcí na základě vstupního mezikódu. Nástroj v rámci tohoto kroku provede standardní úkony v podobě výběru instrukcí, jejich plánování a alokace registrů. Výstupem tohoto kroku pak nebude přímo objektový soubor, ale jak již bylo řečeno, RISC-V/x86 assembly a pro jeho převod do podoby objektového souboru bude využito externích nástrojů.

Překladač by v rámci tohoto kroku měl provést také základní optimalizace jako je constant folding and propagation, dead code elimination, common subexpression elimination a podbně, které nejsou závislé přímo na cílové platformě. Tyto optimalizace by pak z důvodu usnadnění derivace variant pro jiné architektury měly být implementovány na úrovni mezikódu ještě před samotným code generation tak, aby jejich implementace zůstala nezávislá na konkrétní architektuře.

5 Ostatní (mimofunkční) požadavky

5.1 Požadavky na rozšiřitelnost a začlenitelnost

Projekt je koncipován jako nástroj, který bude možné využít jako překladačový backend při tvorbě jednoduchého překladače (doplněním o frontend pro požadovaný jazyk). Z tohoto důvodu je tedy možnost snadné integrace projektu do již existujících/vznikajících projektů jednou z klíčových vlastností. Za tímto účelem projekt definuje vlastní IR standard, který takovouto integraci umožňuje. Pro integraci s projektem pak stačí, aby byl frontend, s nímž chceme integraci provést, upraven tak, aby namísto doposud produkováného výstupu generoval textovou reprezentaci IR definovanou tímto standardem, kterou pak již lze přímo předat jako vstup tomuto projektu.

Ze stejných důvodů hraje důležitou roli i rozšiřitelnost projektu. Bylo by vhodné, aby bylo do budoucna možné projekt rozšířit o další cílové architektury jako například AArch64. Takovýto požadavek na rozšiřitelnost je však mnohem komplikovanější splnitelný, protože ze samotné podstaty problému musí být problém generování kódu (alokace registrů, výběr instrukcí) do nějaké míry závislý na cílové architektuře. Nejlepším možným řešením pro dosažení co nejsnazší rozšiřitelnosti je tedy přesunout co možná nejvíce optimalizací do doby před samotným code generation a aplikovat je ještě na úrovni samotné IR (což ale není vždy možné) a také cílové platformy a generování kódu pro ně vhodně abstrahovat, tak aby jednotlivé implementace mohly sdílet co největší množství existující infrastruktury. Rozšíření o novou podporovanou architekturu pak bude vyžadovat znovu naimplementovat pouze určité pro platformu specifické části code generation bez nutnosti úpravy částí pracujících nad IR. Dalším problémem je v tomto případě závislost na externím assembleru pro generování objektových souborů, který je v takovémto případě třeba nahradit alternativou podporující danou architekturu.

6 Ostatní požadavky

Jak již bylo zmíněno výše, jedním z hlavních požadavků na projekt je jeho rozšiřitelnost, ať už jde o integraci s různými frontendy, nebo implementace dalších cílových platform. V tomto ohledu je tedy třeba dbát na modulární návrh kódu s vhodným rozhraním jednotlivých modulů, tak aby přidání nové cílové platformy v ideálním případě znamenalo pouze přidání nového modulu, který implementuje a poskytuje

pro danou platformu specifické funkce. Integrace s frontendem by pak měla znamenat pouze převedení interní reprezentace překladaného kódu do textového zápisu pomocí vstupní stack-based IR.

7 Negativní vymezení

Zaměřením projektu je čistě generování kódu, tedy vygenerovat RISC-V/x86 assembly odpovídající vstupnímu programu zapsanému pomocí IR. Projekt se tak nezabývá jednotlivými formáty objektových souborů ani jejich generováním. Stejným způsobem se projekt nezaměřuje ani na linkování objektových souborů do podoby spustitelných souborů a formáty těchto souborů používanými jednotlivými operačními systémy. Při řešení těchto podproblémů projekt spoléhá čistě na externí nástroje *as* a *ld*.

8 Time-line & Milestones

Datum	Milník	Způsob prezentace
30.6.2026	Definování podoby IR	Commit/konzultace s vedoucím
31.7.2026	Základní code generation	Commit/konzultace s vedoucím
20.8.2026	Kompletní code generation s optimalizacemi	Commit/konzultace s vedoucím

9 Zakončení

Projekt aktuálně implementuje jednoduchý jednopřechodový code generation pro RISC-V na základě vstupu reprezentujícího vstupní program prostřednictvím stack-based mezikódu. Architektura RISC-V je podporována jak v 64-bitové, tak v 32-bitové verzi (RV64 a RV32) jako základní ukázka mechanismu, kterým bude v budoucnu možné přidat podporu pro další architektury (jako je právě zmíněná x86). Z hlediska podporovaných funkcí code generation aktuálně podporuje pouze celočíselné proměnné a ukazatele, které jsou taktéž reprezentovány jako celé číslo šířky odpovídající velikosti adres cílové platformy. S proměnnými je možné provádět základní aritmetické a logické operace (sčítání, odčítání, násobení, dělení, modulo, porovnání,...). Proměnné typu pointer je pak možné použít k přímé práci s pamětí (load/store), zároveň je možné získat adresu existujících proměnných. Dále je podporováno deklarování a volání funkcí, tedy je implementována většina volací konvence pro RISC-V (s výjimkou ukládání stavu caller-saved registrů). Dále je pak připravována podpora pro globální proměnné a statická data, spolu s přidáním FFI.

9.1 IR

Vstupem samotného code generation kroku je překládaný program zapsaný pomocí projektem definované IR. Prozatím projekt neobsahuje parser textové reprezentace IR, takže je nutné generátoru vstup předávat programaticky z prostředí Haskellu pomocí struktur reprezentujících jednotlivé IR tokeny. Samotná IR je inspirována primárně programovacím jazykem Forth a WebAssembly a jedná se tedy o stack-based reprezentaci. Následující seznam stručně popisuje aktuálně podporované IR tokeny/instrukce.

Podporované IR instrukce

- **IrLiteral** - pushne daný literál na stack
- **Literal** literal_value - vytvoří typovaný literál - aktuálně je podporován pouze typ int, brzy doplněný o string literál
- **Drop** - popne jeden prvek ze zásobníku (a zahodí ho)
- **Branch** target - provede skok na label specifikovaný pomocí target
- **ConditionalBranch** target - popne hodnotu ze zásobníku a pokud je rovna nule, pak provede skok na label specifikovaný pomocí target
- **Label** name - vytvoří label označující začátek nového bloku, na nějž je možné skočit pomocí **Branch/ConditionalBranch**
- **GetLocal** var_name - získá aktuální hodnotu lokální proměnné s daným názvem a pushne ji na zásobník
- **SetLocal** var_name - popne hodnotu ze zásobníku a nastaví ji jako hodnotu dané proměnné
- **GetLocalAddr** var_name - získá adresu/ukazatel na danou proměnnou a pushne ji na zásobník
- **Load** type offset - popne hodnotu ze zásobníku a použije ji jako základní adresu, z níž s daným offsetem načte hodnotu specifikovaného typu (offset se hodí pro práci se strukturami/poli) a pushne ji na zásobník
- **Store** type offset - popne ze zásobníku základní adresu a hodnotu, tuto hodnotu následně uloží na stanovený offset od základní adresy
- **FunctionCall** func_name - popne ze zásobníku n hodnot (kde n je počet argumentů volané funkce) a zavolá specifikovanou funkci, které je předá jako argumenty (pořadí v deklaraci odpovídá pořadí pushování na stacku, jako první argument je předaná nejdříve pushnutá hodnota)
- **Add** - popne dvě čísla ze stacku a pushne zpět jejich součet

- **Sub** - popne dvě čísla ze stacku a pushne zpět jejich rozdíl
- **Mul** - popne dvě čísla ze stacku a pushne zpět jejich součin
- **Div** - popne dvě čísla ze stacku a pushne zpět jejich podíl
- **Mod** - popne dvě čísla ze stacku a pushne zpět zbytek po dělení
- **Lt** - popne dvě čísla ze stacku a porovná je na nerovnost ostře menší než (výsledek pushne na stack stejně jako všechny ostatní operace porovnání, True/False je reprezentováno klasicky jako jedna/nula)
- **Lte** - popne dvě čísla ze stacku a porovná je na nerovnost menší rovno než
- **Gt** - popne dvě čísla ze stacku a porovná je na nerovnost ostře větší než
- **Gte** - popne dvě čísla ze stacku a porovná je na nerovnost větší rovno než
- **Eq** - popne dvě hodnoty ze stacku a porovná je na rovnost (jedná se o identická data)
- **Not** - popne číslo ze stacku a pushne zpět jeho logické not (výsledkem je jedna pro hodnotu nula a nula pro libovolnou jinou hodnotu)

Samotný program je pak v IR reprezentován jako seznam deklarací, žádná z instrukcí výše tak nemůže stát sama o sobě jako top-level výraz, ale musí být součástí deklarace funkce. Deklarace funkce se skládá z prototypu (návratový typ, typy argumentů a název) a definice jejího těla.

V dohledné době k deklaraci funkce jako aktuálně jediné povolené top-level konstrukci v IR přibude ještě deklarace globální proměnné. Spolu s ní pak do IR přibudou operace **GetGlobal**, **SetGlobal**, **GetGlobalAddr** jako alternativy stejnojmenných operací pro práci s lokálními proměnnými funkcí. Dále, jak již bylo zmíněno, bude přidán také nový typ literálu string.

9.2 Implementace code generation

Jak již bylo zmíněno úvodem, samotné generování kódu probíhá aktuálně na jeden průchod IR vstupem a je rozděleno do dvou hlavních modulů. Obecného code generation modulu a modulu poskytujícího implementaci pro konkrétní cílovou architekturu.

Obecný code generation je založený na principu virtuálního zásobníku, který vzhledem k tomu, že projektem zvolená IR je stack-based, za doby kompilace simuluje stav zásobníku překládaného programu. Tento přístup umožňuje poměrně snadno provádět základní lokální optimalizace, jako je constant folding (tedy vyhodnocení operací závislých pouze na konstantních/za překlada známých hodnotách přímo překladačem bez generování jakýchkoliv instrukcí). Nebo při doplnění o jednoduchou cache, která ukládá, jaké hodnoty jsou aktuálně uloženy v jednotlivých registrech (obdoba local value numbering), snížit množství vygenerovaných přístupů do paměti. Obecný code generation modul tak zajišťuje primárně tuto obecnou simulaci virtuálního zásobníku pro jednotlivé IR instrukce a další s ní spojenou správu stavu code generation (invalidace zmíněných cache, udržování známých funkcí a lokálních proměnných,...), která se neliší mezi jednotlivými architekturami. A samotný výběr hardwarových instrukcí a alokaci registrů pak deleguje na modul s konkrétní implementací cílové platformy.

Modul s implementací pak musí za tímto účelem poskytovat implementace dvou parametrizovaných typových tříd definujících rozhraní pro komunikaci s obecným code generation. Těmito třídami jsou *InstSelector* a *RegisterAllocator*. Kdy první z tříd zajišťuje rozhraní pro samotný výběr hardwarových instrukcí a druhá pro alokaci registrů procesoru. V obecnosti by mělo být do budoucna možné dohromady používat různé parametrizovaný instruction selector a register allocator (vazbu mezi třídami sive nelze odstranit úplně, protože obě do nějaké míry závisí na konkrétní cílové architektuře, ale bylo by možné například umožnit dynamicky přepínat mezi různými algoritmy alokace registrů pro stejnou instrukční sadu). V současné chvíli ale implementace spoléhá na fakt, že pro každou platformu existuje jeden instruction selector a k němu odpovídající register allocator.

Posledním krokem, který už příliš nesouvisí se samotným generováním kódu, je převod instrukcí vygenerovaných code generation krokem do podoby tisknutelného assembly. Převod interní reprezentace instrukcí pomocí algebraických/zobecněných algebraických datových typů do textové podoby je pak opět implementován pomocí parametrizované typové třídy *AsmPrinter*, která každé instrukci přiřadí její textovou reprezentaci. Tento design pak umožňuje v budoucnu implementovat více různých parametrizovaných instancí dané třídy a poměrně snadno tak získat podporu pro více různých syntaxí zápisu stejné instrukční sady (jako je třeba Intel a GNU syntaxe v případě x86).

9.3 Demo

Funkcionalitu doposud naimplementovanou v rámci projektu lze vyzkoušet prostřednictvím poskytovatelného CLI. Jak již bylo zmíněno výše, projekt v tuto chvíli postrádá plnohodnotný parser textové reprezentace mezikódu. Z tohoto důvodu prozatím jednotlivé vzorové ukázky užití spoléhají na serializaci vstupu ve formátu *json* pomocí Haskell knihovny *aeson*. Ukázkové programy zapsané v IR serializované do formátu *json* lze nalézt v repozitáři projektu v podadresáři *resources/*. Konkrétně se jedná o ukázku rekurzivního výpočtu Fibonacciho čísel v *fibonacci.json* a iteračního výpočtu faktoriálu ve *factorial.json*. Obě ukázky pak pro jednoduchost vypočtenou hodnotu vrací jako exit code celého programu.

Pro přeložení poskytovaných ukázek je nejprve třeba sestavit samotný překladač pomocí `cabal build`. Výsledný spustitelný soubor pak nabízí následující jednoduché konzolové rozhraní.

Konzolové rozhraní

- demo-file, -d** – slouží pro specifikování cesty k *.json* souboru s definicí překládaného programu
- output, -o** – slouží pro specifikování výstupního souboru (výchozí hodnota: *out.s*)
- target-arch, -a** – slouží pro výběr cílové architektury *Rv32|Rv64* (výchozí hodnota: *Rv32*)

Pro přeložení ukázky výpočtu faktoriálu tak následně stačí spustit výsledný spustitelný soubor následujícím způsobem (příklad předpokládá, že ke spouštění dochází z kořenového adresáře projektu a *codegen* sem byl přesunut z build složky, jinak je třeba upravit cesty k souborům odpovídajícím způsobem). Příklad s výpočtem Fibonacciho čísel lze přeložit analogicky.

```
./codegen -d ./resources/factorial.json -o factorial.s -a Rv32
```

Vzhledem k omezenému rozšíření v současnosti jediné podporované platformy RISC-V bude pro spuštění vygenerovaného assembly pravděpodobně nutné využít některý z dostupných emulátorů/simulátorů. Nejjednodušší cestou je v tomto ohledu využít některý z webových simulátorů, do kterých ve většině případů stačí pouze zkopírovat obsah výstupního souboru. Případně je možné použít lokální emulátor, jako je například QEMU. Druhá z metod je z dlouhodobého hlediska spolehlivější, ale vyžaduje instalaci jak samotného emulátoru, tak i *GNU toolchainu* pro RISC-V na koncové zařízení, aby bylo možné vygenerované assembly sestavit a nalinkovat pomocí *as* a *ld*.

A Vymezení pojmů

IR - Zjednodušený mezikód, na nějž je překládaný kód převeden frontendem překladače, a slouží jako univerzální na jazyce nezávislá reprezentace a rozhraní mezi backendem a frontendem překladače.